

Prompt Manager Project 면접 대비 답변 정리

1. 코드 구조 및 설계

1-1. 기능별로 함수를 분리한 기준과 각 함수의 역할을 설명해 주세요.

기능별로 함수를 분리한 기준은 **하나의 함수가 하나의 역할만 담당하도록** 하는 것입니다.

메뉴 출력, 프롬프트 추가, 목록 조회, 검색, 상세 보기처럼 사용자 입장에서 서로 다른 기능을 각각 별도의 함수로 만들었습니다.

주요 함수의 역할은 다음과 같습니다.

- `show_menu()`
메뉴를 화면에 출력하는 함수입니다.
- `add_prompt()`
제목, 내용, 카테고리를 입력받아 새로운 프롬프트를 추가하는 함수입니다.
- `show_prompt_list()`
저장된 전체 프롬프트를 번호와 함께 출력하는 함수입니다.
- `show_category()`
선택한 카테고리에 해당하는 프롬프트만 출력하는 함수입니다.
- `search_prompt()`
제목 또는 내용에 검색어가 포함된 프롬프트를 찾는 함수입니다.
- `show_prompt_detail()`
선택한 프롬프트의 전체 정보를 출력하는 함수입니다.
- `manage_favorite()`
프롬프트의 즐겨찾기 상태를 추가하거나 해제하는 함수입니다.
- `show_favorites()`
즐거찾기로 등록된 프롬프트만 출력하는 함수입니다.
- `main()`
메뉴를 반복해서 보여주고, 사용자의 선택에 따라 각 기능 함수를 호출하는 함수입니다.

이렇게 함수를 분리하면 코드를 읽기 쉽고, 특정 기능을 수정할 때 다른 기능에 미치는 영향을 줄일 수 있습니다.

한 문장 답변

기능별로 함수를 분리했고, 각 함수는 하나의 기능만 담당하도록 설계했습니다.

기능마다 하는 일이 다르기 때문에 각각 함수로 나눴습니다.

예를 들어 `show_menu()` 는 메뉴를 출력하고, `add_prompt()` 는 프롬프트를 추가하며, `search_prompt()` 는 검색을 담당합니다.

이렇게 나누면 코드가 읽기 쉽고, 문제가 생겼을 때 해당 함수만 수정할 수 있습니다.

1-2. 프롬프트 데이터 구조의 필드 구성과 접근 방식을 설명해 주세요.

여러 개의 프롬프트는 하나의 리스트에 저장하고, 프롬프트 하나는 딕셔너리로 표현했습니다.

```
prompts= [
    {"title": "회의록 요약", "content": "회의 내용을 핵심 요약...", "category": "텍스트 생성", "favorite": True}
]
```

각 필드의 역할은 다음과 같습니다.

- `title`
프롬프트 제목
- `content`
프롬프트 전체 내용
- `category`
프롬프트 분류
- `favorite`
즐거찾기 여부

리스트 전체를 반복할 때는 다음과 같이 작성합니다.

```
for prompt in prompts: print(prompt["title"])
```

특정 프롬프트의 제목이나 카테고리를 가져올 때는 딕셔너리의 키를 사용합니다.

```
prompt["title"] prompt["category"]
```

리스트는 여러 개의 프롬프트를 순서대로 저장하고, 딕셔너리는 프롬프트 한 개의 여러 속성을 이름으로 관리합니다.

한 문장 답변

리스트는 여러 프롬프트를 저장하고, 딕셔너리는 프롬프트 하나의 제목, 내용, 카테고리, 즐겨찾기 정보를 저장합니다.

1-3. 입력 검증 로직을 어디에 두었고 어떻게 동작하게 했나요?

입력 검증이란 **사용자가 입력한 값이 올바른지 확인하는 과정**입니다.

프로그램은 사용자가 항상 정확하게 입력할 것이라고 가정할 수 없기 때문에, 잘못된 입력을 먼저 검사해야 합니다.

예를 들어,

- 제목을 입력하지 않고 엔터만 누르는 경우
- 존재하지 않는 프롬프트 번호를 입력하는 경우

이런 상황을 그대로 처리하면 프로그램이 잘못 동작하거나 오류가 발생할 수 있습니다.

그래서 입력을 받은 후 **올바른 값인지 먼저 확인하는 과정**이 필요합니다.

이 프로그램에서는 크게 두 가지를 검증했습니다.

1. 빈 값 검증

프롬프트를 추가할 때

- 제목
- 내용

이 비어 있으면 저장하지 않고 다시 입력받도록 구현했습니다.

예를 들어 사용자가

제목 :

에서 아무것도 입력하지 않고 엔터를 누르면

제목을 입력해주세요.

라는 안내 메시지를 출력하고 다시 입력을 요청합니다.

2. 번호 범위 검증

상세 보기나 즐겨찾기 기능에서는 사용자가 프롬프트 번호를 입력합니다.

예를 들어 저장된 프롬프트가 3개인데

```
1
2
3
```

사용자가

```
7
```

을 입력하면 존재하지 않는 번호입니다.

이 경우에는

```
올바른 번호를 입력해주세요.
```

라는 안내 메시지를 출력하고 다시 입력하도록 구현했습니다.

입력 검증은 **입력을 받는 각 함수 안에서 바로 수행하도록 구현했습니다.**

예를 들어

`add_prompt()`

제목과 내용을 입력받는 함수이므로

- 빈 값인지 검사

를 이 함수 안에서 수행합니다.

`show_prompt_detail()`

번호를 입력받는 함수이므로

- 번호가 리스트 범위 안에 있는지 검사

를 이 함수 안에서 수행합니다.

즉,

| 입력을 받는 곳에서 바로 검증하도록 설계했습니다.

빈 값을 검사할 때는 `while` 반복문을 사용했습니다.

```
while True: title = input("제목: ").strip() if title: break print("제목을 입력해주세요.")
```

동작 순서는 다음과 같습니다.

1. 제목을 입력받습니다.
2. `strip()` 으로 앞뒤 공백을 제거합니다.
3. 입력한 값이 있는지 확인합니다.
4. 값이 있으면 `break` 로 반복문을 종료합니다.
5. 값이 없으면 안내 메시지를 출력하고 다시 입력받습니다.

즉,

사용자가 올바른 값을 입력할 때까지 계속 반복합니다.

`strip()` 은 문자열의 앞뒤 공백을 제거하는 함수입니다.

예를 들어

```
회의록 요약
```

을 입력하면

```
회의록 요약
```

으로 바꿔줍니다.

더 중요한 이유는

사용자가 스페이스바만 여러 번 입력한 경우에도

```
" "
```

을

```
" "
```

빈 문자열로 만들어 주기 때문입니다.

그래서 실제로 아무 내용도 입력하지 않았는지 정확하게 확인할 수 있습니다.

`while` 은 조건을 만족할 때까지 계속 반복하는 반복문입니다.

입력이 잘못되었을 때

한 번만 검사하는 것이 아니라

올바른 값을 입력할 때까지 계속 다시 입력받기 위해 사용했습니다.

예를 들어

제목 입력

↓

빈 값 입력

↓

다시 입력

↓

빈 값 입력

↓

다시 입력

↓

정상 입력

↓

반복 종료

이런 방식으로 동작합니다.

`break` 는 반복문을 종료하는 명령어입니다.

사용자가 정상적인 제목을 입력하면

`break`

를 실행하여 반복문을 끝내고

다음 코드로 넘어갑니다.

번호는 다음과 같이 검사했습니다.

```
if 1<=number<=len(prompts):
```

이 의미는

입력한 번호가 1 이상이고,
저장된 프롬프트 개수 이하인지 확인하는 것입니다.

예를 들어

프롬프트가 3개라면

```
len(prompts)
```

의 결과는

```
3
```

입니다.

따라서

```
1<=number<=3
```

인지 검사합니다.

조건을 만족하면 정상 처리하고,

조건을 만족하지 않으면

```
올바른 번호를 입력해주세요.
```

를 출력합니다.

`len()` 은 리스트에 저장된 데이터의 개수를 알려주는 함수입니다.

예를 들어

```
prompts= [
    {...},
    {...},
    {...}
]
```

라면

```
len(prompts)
```

의 결과는

```
3
```

입니다.

이 값을 이용하여 사용자가 입력한 번호가 범위를 벗어났는지 검사했습니다.

사용자는 번호를

```
1
2
3
```

으로 보지만,

리스트의 인덱스는

```
0
1
2
```

부터 시작합니다.

따라서

사용자가

```
1
```

을 입력하면

실제로는

```
prompts[0]
```

을 가져와야 합니다.

그래서

```
prompts[number-1]
```

처럼 작성했습니다.

1-4. 커밋을 기능 단위로 나눈 기준을 설명해 주세요.

커밋은 하나의 기능이 독립적으로 완성되는 시점을 기준으로 나눴습니다.

예시는 다음과 같습니다.

```
chore: initialize project structure
feat: add main menu
feat: add prompt
feat: add category filter
feat: add search feature
feat: add prompt detail
feat: add favorite management
feat: add favorite list
docs: update README
docs: add screenshots
```

메뉴 구현, 검색 기능, 즐겨찾기 기능처럼 서로 다른 목적의 변경사항을 한 커밋에 섞지 않았습니다.

이렇게 하면 어떤 기능이 언제 추가되었는지 쉽게 알 수 있고, 문제가 발생했을 때 해당 기능의 변경 이력만 확인하거나 이전 상태로 되돌리기 쉽습니다.

한 문장 답변

독립된 기능 하나가 완성되고 정상 작동을 확인한 시점마다 커밋했습니다.

2. 핵심 기술 원리 적용

2-1. 리스트와 딕셔너리를 선택한 이유와 장단점을 설명해 주세요.

리스트는 여러 개의 프롬프트를 순서대로 저장하기 위해 선택했습니다.

리스트의 장점

- `append()` 로 새로운 데이터를 쉽게 추가할 수 있습니다.
- 반복문으로 전체 목록을 쉽게 조회할 수 있습니다.
- 인덱스를 이용해 번호 기반 상세 조회가 가능합니다.

리스트의 단점

- 데이터가 많아질수록 검색할 때 처음부터 끝까지 반복해야 해서 속도가 느려질 수 있습니다.
- 특정 항목을 바로 찾기 위해서는 반복문이 필요한 경우가 많습니다.

딕셔너리는 프롬프트 하나가 제목, 내용, 카테고리, 즐겨찾기처럼 여러 속성을 가지기 때문에 선택했습니다.

딕셔너리의 장점

- `prompt["title"]` 처럼 필드 이름으로 접근할 수 있습니다.
- 각 값의 의미를 쉽게 이해할 수 있습니다.
- 새로운 필드를 추가하기 쉽습니다.

딕셔너리의 단점

- 키 이름을 잘못 작성하면 오류가 발생할 수 있습니다.
- 모든 프롬프트가 같은 키 구조를 갖도록 관리해야 합니다.

한 문장 답변

리스트는 여러 프롬프트를 관리하기 좋고, 딕셔너리는 프롬프트 하나의 여러 속성을 이름으로 관리하기 좋기 때문에 함께 사용했습니다.

2-2. 메뉴 반복에 while을 사용한 이유와 종료 조건을 설명해 주세요.

사용자가 한 번 기능을 실행한 뒤에도 계속 다른 기능을 선택할 수 있어야 하므로 `while` 반복문을 사용했습니다.

```
while True: show_menu() choice = input("선택: ") if choice == "0": print("프로그램을 종료합니다.") break
```

`while True` 는 프로그램을 계속 반복하게 합니다.

사용자가 종료 번호인 `0` 을 입력하면 `break` 를 실행하여 반복문을 끝냅니다.

각 기능이 종료되면 다시 반복문의 처음으로 돌아가 메뉴가 다시 출력됩니다.

잘못된 번호를 입력해도 프로그램이 종료되지 않고 안내 메시지를 출력한 뒤 다시 메뉴로 돌아갑니다.

한 문장 답변

메뉴를 계속 반복하기 위해 `while True` 를 사용했고, 사용자가 0을 입력하면 `break` 로 종료하도록 설계했습니다.

2-3. 검색 기능에서 제목 또는 내용 포함을 어떻게 구현했나요?

사용자로부터 검색어를 입력받은 뒤 모든 프롬프트를 반복하면서 제목 또는 내용에 검색어가 포함되어 있는지 확인했습니다.

```
keyword = input("검색어: ").strip().lower() results = [] for prompt in prompts: title = prompt["title"].lower() content = prompt["content"].lower() if keyword in title or keyword in content: results.append(prompt)
```

핵심은 `in` 연산자와 `or` 조건입니다.

- `keyword in title`
검색어가 제목에 포함되어 있는지 확인합니다.
- `keyword in content`
검색어가 내용에 포함되어 있는지 확인합니다.
- `or`
제목이나 내용 중 하나에만 검색어가 포함되어 있어도 검색 결과에 추가합니다.

`lower()` 를 사용하여 영문 대소문자 차이 때문에 검색 결과가 누락되지 않도록 했습니다.

검색 결과는 별도의 리스트인 `results` 에 저장한 뒤 출력했습니다.

한 문장 답변

모든 프롬프트를 반복하면서 `검색어 in 제목 or 검색어 in 내용` 조건으로 포함 여부를 확인했습니다.

2-4. 브랜치를 분리해서 작업한 이유와 병합 기준을 설명해 주세요.

새로운 기능을 main 브랜치에서 바로 개발하면 작업 중인 코드 때문에 안정적인 버전이 깨질 수 있습니다.

그래서 프롬프트 목록 기능을 별도의 `feature/prompt-list` 브랜치에서 작업했습니다.

브랜치 생성 명령어는 다음과 같습니다.

```
git checkout -b feature/prompt-list
```

기능을 구현하고 테스트한 뒤 커밋했습니다.

```
git add .git commit -m "feat: add prompt list"
```

그다음 main 브랜치로 이동했습니다.

```
git checkout main
```

기능이 정상 작동하고 커밋까지 완료된 시점에 병합했습니다.

```
git merge feature/prompt-list
```

병합 기준

- 요구 기능이 모두 구현되었는가
- 잘못된 입력도 처리되는가
- 기존 기능이 정상 작동하는가
- 기능 단위 커밋이 완료되었는가

한 문장 답변

안정적인 main 브랜치를 보호하기 위해 기능 브랜치에서 작업했고, 구현과 테스트와 커밋이 완료된 뒤 main에 병합했습니다.

3. 심층 인터뷰

3-1. 프로그램을 종료해도 데이터가 유지되게 하려면 어떤 설계가 필요한가요?

현재 프로그램은 프롬프트 데이터를 메모리의 리스트에만 저장하기 때문에 프로그램이 종료되면 데이터가 사라집니다.

데이터를 유지하려면 프로그램 시작 시 파일에서 데이터를 불러오고, 데이터가 변경될 때 파일에 저장하는 구조가 필요합니다.

저는 JSON 파일 형식을 선택하겠습니다.

JSON을 선택하는 이유

- Python의 리스트와 딕셔너리 구조와 비슷합니다.
- 사람이 직접 읽기 쉽습니다.
- Python 기본 라이브러리인 `json` 모듈로 처리할 수 있습니다.
- 외부 라이브러리가 필요하지 않습니다.

구조는 다음과 같이 설계할 수 있습니다.

```
def load_prompts(): # 프로그램 시작 시 JSON 파일을 읽는 함수
pass
def save_prompts(): # 데이터가 변경될 때 JSON 파일에 저장하는 함수
pass
```

프로그램을 시작할 때 `load_prompts()` 를 호출합니다.

프롬프트 추가, 수정, 삭제, 즐겨찾기 상태 변경 후에는 `save_prompts()` 를 호출합니다.

파일이 존재하지 않거나 파일 내용에 문제가 있는 경우에는 기본 데이터로 시작하도록 예외 처리도 필요합니다.

한 문장 답변

프로그램 시작 시 JSON 파일을 불러오고, 데이터가 변경될 때마다 다시 저장하는 구조로 만들겠습니다.

3-2. 같은 제목의 프롬프트가 여러 개 생긴다면 어떻게 처리하겠습니까?

가장 안전한 방법은 제목을 고유한 기준으로 사용하지 않고, 각 프롬프트에 별도의 `id`를 부여하는 것입니다.

```
{ "id": 1, "title": "회의록 요약", "content": "...", "category": "텍스트 생성", "favorite": false }
```

같은 제목이 등록되더라도 `id`가 다르면 서로 다른 프롬프트로 구분할 수 있습니다.

추가할 때 같은 제목이 이미 존재한다면 사용자에게 경고를 보여줄 수 있습니다.

같은 제목의 프롬프트가 존재합니다.

1. 그래도 추가
2. 기존 프롬프트 수정
3. 취소

같은 제목이라고 해서 무조건 추가를 막는 것보다는, 사용자에게 선택권을 주는 방식이 좋습니다.

제목은 같지만 내용이나 카테고리가 다를 수도 있기 때문입니다.

한 문장 답변

같은 제목은 허용하되 고유한 ID로 구분하고, 등록 시 중복 제목이 있다는 경고를 보여주세요.

3-3. 병합 과정에서 충돌이 발생하면 어떤 순서로 해결하겠습니까?

먼저 `git status`를 사용하여 충돌이 발생한 파일을 확인합니다.

```
git status
```

그다음 충돌 파일을 열어 Git이 표시한 충돌 구간을 확인합니다.

```
<<<<<< HEAD
main 브랜치의 코드
=====
feature 브랜치의 코드
>>>>>> feature/prompt-list
```

양쪽 코드를 비교한 뒤 필요한 코드를 선택하거나 두 내용을 올바르게 합칩니다.

수정 후에는 충돌 표시인 <<<<<<, =====, >>>>>> 를 모두 제거합니다.
그다음 프로그램을 직접 실행하여 기능이 정상적으로 작동하는지 확인합니다.
검증이 끝나면 수정한 파일을 다시 스테이징합니다.

```
git add prompt_manager.py
```

마지막으로 충돌 해결 내용을 커밋합니다.

```
git commit-m"fix: resolve merge conflict"
```

필요하면 다음 명령어로 병합 이력도 확인합니다.

```
git log--oneline--graph
```

병합 충돌 해결 순서

1. git status로 충돌 파일 확인
2. 양쪽 브랜치의 변경 내용 비교
3. 필요한 코드 선택 또는 통합
4. 충돌 표시 제거
5. 프로그램 실행 및 기능 테스트
6. git add
7. git commit
8. git status와 git log로 최종 확인

한 문장 답변

충돌 파일을 확인하고 양쪽 변경 내용을 비교해 수정한 뒤, 실행 테스트를 거쳐 add와 commit으로 충돌 해결을 완료합니다.

3-4. 카테고리 체계가 변경되거나 늘어난다면 어느 부분을 수정하는 것이 가장 안전한가요?

카테고리 목록을 여러 함수에 직접 반복해서 작성하면, 카테고리가 변경될 때 여러 곳을 수정해야 하므로 실수할 가능성이 높습니다.

따라서 카테고리를 하나의 공통 리스트로 관리하는 것이 가장 안전합니다.

```
CATEGORIES= ["텍스트 생성", "이미지 생성", "영상 생성", "페르소나", "자동화", "기타"]
```

프롬프트 추가 함수와 카테고리 조회 함수가 모두 이 리스트를 사용하도록 설계합니다.

```
for index, category in enumerate(CATEGORIES, start=1): print(f"{index} {category}")
```

새로운 카테고리를 추가할 때는 `CATEGORIES` 리스트 한 곳만 수정하면 됩니다.

기존 JSON 데이터가 있다면 이전 카테고리 이름과 새로운 카테고리 이름이 달라졌을 때 기존 데이터도 함께 변경해야 합니다.

예를 들어 `"텍스트"` 를 `"텍스트 생성"` 으로 변경한다면 기존 데이터의 카테고리 값도 함께 변환해야 합니다.

한 문장 답변

카테고리 목록을 하나의 공통 상수 리스트로 관리하고 모든 함수가 그 값을 참조하도록 만드는 것이 가장 안전합니다.